Name : Rajesh Singh

Collaborators / External Res : None

No. of late days on prev pset = 0

No. of late days used after this pset = 0

1(a)

f	g	Ω	ω	Θ
$3^{(\log_2 n)^2}$	$2(\ln n)\cdot(\ln n+3)$	T	F	T
$3n3$	$\lvert\{S \in [n] : \lvert S\rvert \le 5\}\rvert$	T	F	F
$(n+1)^{n+1}$	$(n+1)\times n!$	T	F	F
4^n	$(3+(-1)^n)^n$	T	F	F

1(b) For all $n \in \mathbb{N}$ $T^{=}(n) \le T(n)$

if $T^{=}(n) = \Omega(g)$

$\Rightarrow$ $T^{=}(n) \ge c(g)$ since Ω = lower bound.

where $c > 0$ for $n > n_0$

since $T(n) \ge T^{=}(n) \ge c(g)$

$\Rightarrow$ $T(n) \ge c(g)$

Hence $T(n) = \Omega(g)$

So, statement, $T^{=}(n) = \Omega(g) \Rightarrow T(n) = \Omega(g)$ is True.

$f(n) = 4^n$, $g(n) = (3+(-1)^n)^n$ $T^{=}(1) = 2$ $T^{=}(2) = 16$ $T^{=}(3) = 8$

$T(n) = 16$ (max of (1, 2, 3))

$\Rightarrow$ max of (1, 2, 3)

Contd....

1(b) False statement: $T = \Omega(g) \Rightarrow T = \Omega(g)$

This can be proved with $g(n) = (3 + (-1)^n)^n$

if $T = \Omega(g) \Rightarrow T \geq c \cdot g(n)$

assuming $c = 1$ $T \geq 2^n$ (for n is odd)

Now, $T^2(1) = 2$, $T^2(2) = 4^2 = 16$

but T is $2^2 = 4$

so ↯ contradicts and

hence statement is false.

2(a) $I = (n, b, k) = (57, 12, 3)$

$c_0 = 57 \bmod 12 = 9$, $n = \frac{57-9}{12} = 4$

$c_1 = 4 \bmod 12 = 4$, $n = \frac{4-4}{12} = 0$

$c_2 = 0 \bmod 12 = 0$

So, algorithm returns $\boxed{(9, 4, 0)}$

Yes, ISC output is valid answer for TT

since $f(57, 12, 3) = \{(9, 4, 0) : n = 9 + 4 \cdot 12 + \ldots$

$n = 57$

and $\forall i \; 0 < c_i < 12$

$c_i = (9, 4, 0)$

2(b) For $x : (n, b, k) = (57, 10, 3)$

Output of BC is $(7, 5, 0)$

which is reverse digit of 057
i.e, n.

So, problem Π is: given integer n and $b \geq 2$ and number of digit k, express n using exactly k base-b digits $(c_0, c_1, \ldots c_{k-1})$ listed from least to most significant digit if n cannot fit in k digits then return $\perp$; so no output.

2(c) $x = (10, 2, 2)$, the 2 binary digit can represent only upto $2^2 - 1 = 3$
so in this case $f(x) = \phi$
since there is no valid output.

2(d) For valid output, if condition need to satisfy i.e, $(n == 0)$ and that can happen only when #digit in $n \leq k$
so, $|f(x)| = k$ where $x = (n, b, k)$

2(e) No, $x = (10, 2, 2)$ $f(x) = \perp$
but $f'(x) = \phi \cup \{0, 1\} \neq \perp$
so Π' will return $(0, 1)$
but Π will return $\perp$
So using counterexample, it's proved. $\Pi \neq \Pi'$

3(a) (proving correctness of algorithms)
- Radix Sort.

Let V_i' be the digit array produced by BC for an original key K_i and let V_i be it original value.

$k = \lceil \log_b U \rceil$ $U \in N$ is universe size

loop invariant: After j passes, the records are sorted by number represented by their j least significant digits

~~Every original on~~

Base case $j = 0$

before any passes, 0 digit have been process
So every number is 0 since # digit is 0.
so initial ordering satisfy invariant.

Inductive Step.

Assume the invariant holds after j passes where $j < k$. The next pass sorts the array by digit j. - next more significant digit
There could be two condition:

- digit are different at position j:
  Singleton Bucket sort places smaller digit first. This also correctly orders their lower digit at position j.

- digit at position j are equal. Here stability preserves their previous order. ~~By induction~~
  By Induction hypothesis, the order is already sorted by their lowest j digits since them next digit are equal, their lowest j+1 digits are also correctly sorted.

Thus after next pass, the records are sorted by their lowest j+1 digits.

Finally, after all k passes the invariant gaurntees that alll the keys are sorted by their lowest k digits

final loop reconstruction, each key is created from V'

$$K_i = V_i'[0] + V_i'[1] \cdot b + \ldots + V_i'[k-1] \cdot b^{k-1}$$

Also each value is associated with key this restores the original key-value pair.

3(b) Radix Sort (Analyzing runtime)

$k = \lceil \log_b U \rceil$ = max number of digits in a number of Universe $U \in \mathbb{N}$ with number of base b, and each digit in range 0 to $k-1$.

Radix algorithm uses selection sort on k digits
Complexity of selection sort for single column in a digit is $O(n+b)$
so for k digits it $O((n+b)\cdot k)$

$$= O((n+b)\lceil \log_b U \rceil)$$

Assuming $b = \min\{n, U\} \geq 2$, considering

Case 1: $U \geq n$, $b = n$, this means

$$T = O(n+n)\log_n U = O(n)\frac{\log U}{\log n}$$

Case 2: $U < n$, then $b = U$ thus means

$$T = O(n+U)\log_U U$$
$$= O(n+U) = O(n) \quad (\text{since } U < n)$$

Combining both cases:

$$T = O\left(n + \log n \cdot \frac{\log U}{\log n}\right)$$